

TECHNICAL REPORT

Securing and Scaling a Production-Style Web Infrastructure

Web Infrastructure • Summative Assignment

THE ROOTERS

GROUP 06

GROUP MEMBERS

Hildegardine IHIRWE

Lovella NKUSI TETA

Ange Claire UWINEZA

Elvire AKAYEZU

Module: Web Infrastructure

Submission Date: 19 September 2026

Click GitHub Repository_link: [The Rooters Secure Web Infrastructure](#)

1. Introduction

This report documents how our group, The Rooters, designed, configured, tested and secured a small production style web infrastructure on a single Linux environment. The work combines a reverse proxy and load balancer (NGINX), encrypted communication (self signed TLS/SSL), controlled network exposure (a UFW firewall), and evidence led troubleshooting of a deliberate fault. Every configuration described below was written, run and tested in our own environment; command outputs and screenshots throughout this report are our own evidence.

1.1 Environment Identification

Before configuring any service, we confirmed the identity and software versions of the Linux environment we would be working in throughout the project. This establishes a baseline and ensures every command, configuration file and screenshot referenced later in this report can be traced to a specific, verifiable machine and user context.

```
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ whoami
thirwe
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ pwd
/home/thirwe/the_rooters_web_summative
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ hostname
thirwe-HP-EliteBook-840-G5
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ hostname -I
172.17.0.154
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ python3 --version
Python 3.12.3
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ nginx -v
nginx version: nginx/1.24.0 (Ubuntu)
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ ufw --version
ufw 0.36.2
Copyright 2008-2023 Canonical Ltd.
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl --version
curl 8.5.0 (x86_64-pc-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0.21.2 (+libidn2/2.3.7) libssh/0.10.6/openssl/zlib nghttp2/1.59.0 librtmp/2.3 OpenLDAP/2.6.7
Release-Date: 2023-12-06, security patched: 8.5.0-2ubuntu10.13
Protocols: dict file ftp ftps gopher gophers http https imap ldaps mqtt pop3 pop3s rtmp rtsp scp sftp smb smbs smtp smtps telnet tftp
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM PSL SPNEGO SSL threadsafe TLS-SRP UnixSockets zstd
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$
```

whoami confirms the Linux user account under which all subsequent commands were executed. pwd records the working directory used as the base for our project files. hostname and hostname -I identify the machine name and its local IP address, which is later relevant when discussing network exposure and firewall testing. uname -a records the kernel and system architecture, useful for reproducibility. python3 --version, nginx -v, openssl version, ufw --version and curl --version confirm that every tool required for the assignment (backend services, reverse proxy, TLS, firewall, and testing) was installed and available before configuration began.

Where a required tool was missing, it was installed with:

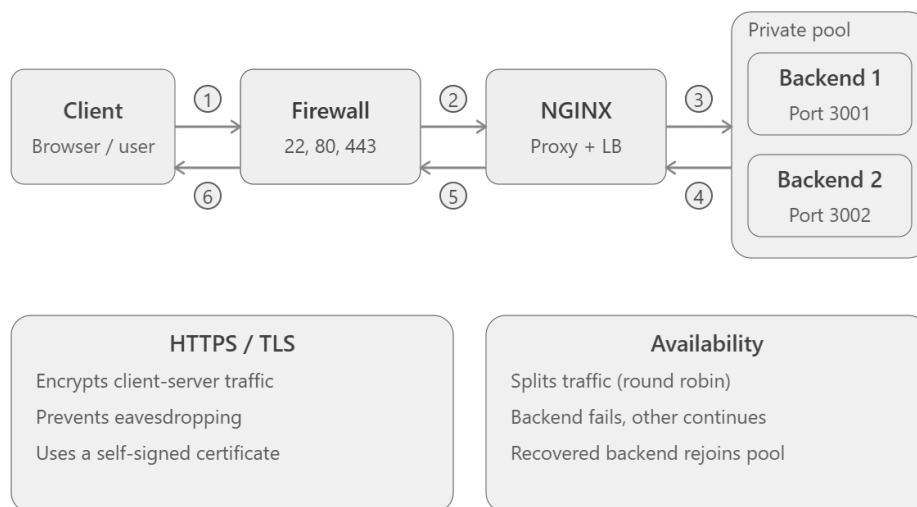
```
sudo apt update
```

```
sudo apt install nginx ufw curl openssl -y
```

sudo was required here because installing packages, editing files under /etc/nginx, managing system services and writing TLS certificate files under /etc/ssl all require administrative privileges that a standard user account does not have by default.

2. Architecture

The diagram below shows the full request path: a client reaches the system only through the firewall and NGINX. NGINX is the single public entry point; it terminates HTTPS, decides which of the two backend services should answer, and forwards the request. The backend services listen only on 127.0.0.1 and are never exposed directly to the network.



Request path, in order

- Client sends a request on HTTP :80.
- The firewall allows the connection (port 80 is one of the three permitted ports).
- NGINX responds with an HTTP 301 redirect to the HTTPS address on :443.
- The client resends the request over HTTPS :443; the firewall allows this too.
- NGINX terminates TLS and forwards the request to whichever backend (127.0.0.1: 3001 or 127.0.0.1:3002) is next in the load-balancing rotation.

- The backend's response returns through NGINX, then through the firewall, back to the client.

Clients never address a backend port directly. This matters for two reasons: it keeps HTTPS and firewall protection in front of every request, and it means the backend pool can lose one member without the client facing service going down.

3. Backend Environment

We built two minimal Python HTTP services using the built in `http.server` module, each listening on its own local port and returning a distinct plain text message so that, once NGINX is in front of them, we can prove which backend actually answered a given request.

Backend 1 (127.0.0.1:3001)

```
from http.server import BaseHTTPRequestHandler, HTTPServer

class RqstHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        message = "Welcome to server 1 of our group the _rooters."
        self.send_response(200)
        self.send_header("Content-Type", "text/plain")
        self.end_headers()
        self.wfile.write(message.encode())

server = HTTPServer(("127.0.0.1", 3001), RqstHandler)
print("Server 1 running on 127.0.0.1:3001")
server.serve_forever()
```

Backend 2 (127.0.0.1:3002)

```
from http.server import BaseHTTPRequestHandler, HTTPServer
```

```
class RqstHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        message = "Welcome to server 2, of our group the_rooters."
        self.send_response(200)
        self.send_header("Content-Type", "text/plain")
        self.end_headers()
        self.wfile.write(message.encode())

server = HTTPServer(("127.0.0.1", 3002), RqstHandler)
print("Server 2 running on 127.0.0.1:3002")
server.serve_forever()
```

Identical in structure, listening on port 3002 and returning the message "Welcome to the server 2, of our group the_rooters" so the two responses are clearly distinguishable.

Why the responses must be distinguishable

If both backends returned identical content, a client (or NGINX itself) would have no way to prove which server actually handled a given request. Distinct messages are what let us demonstrate, later, that NGINX is genuinely alternating between two real, independent processes rather than always using one.

Running and verifying the backends

```
python3 backend1.py    # Terminal 1
python3 backend2.py    # Terminal 2

curl http://127.0.0.1:3001
curl http://127.0.0.1:3002
ss -ltnp | grep -E '3001|3002'
```

```

ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ python3 backend1.py
Server 1 running on 127.0.0.1:3001
127.0.0.1 - - [12/Sep/2026 12:28:04] "GET / HTTP/1.1" 200 -

```

Figure 1: Backend 1 startup and request log. This shows the service starting and printing "Server 1 running on 127.0.0.1:3001", followed by its access log entry for an incoming GET request that received an HTTP 200 response.

```

ihirwe@ihirwe-HP-EliteBook-840-G5:~$ cd ~/the_rooters_web_summative/
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ python3 backend2.py
Server 2 running on 127.0.0.1:3002
127.0.0.1 - - [12/Sep/2026 12:28:50] "GET / HTTP/1.1" 200 -

```

Figure 2: Backend 2 startup and request log. This shows the service starting and printing "Server 2 running on 127.0.0.1:3002", followed by its access log entry for an incoming GET request that received an HTTP 200 response.

```

Sep 12 12:31
ihirwe@ihirwe-HP-EliteBook-840-G5: ~
ihirwe@ihirwe-HP-EliteBook-840-G5:~$ curl http://127.0.0.1:3001
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~$ curl http://127.0.0.1:3002
Welcome to the server 2, of our group the_rooters
ihirwe@ihirwe-HP-EliteBook-840-G5:~$ ss -ltnp | grep -E '3001|3002'
LISTEN 0      5          127.0.0.1:3002      0.0.0.0:*        users:(("python3",pid=7933,fd=3))
LISTEN 0      5          127.0.0.1:3001      0.0.0.0:*        users:(("python3",pid=7927,fd=3))
ihirwe@ihirwe-HP-EliteBook-840-G5:~$

```

Figure 3: ss -ltnp output showing both ports 3001 and 3002 listening, owned by our python3 processes.

What this proves: Both backends are running as independent processes on their expected ports and return distinguishable, correct responses; this is the baseline the load-balancing tests in Section 4 build on.

4. NGINX Reverse Proxy and Load Balancing

NGINX was installed and configured as the single web entry point in front of both backend services. The upstream block groups the two backends into a named pool; NGINX distributes incoming requests across that pool and forwards (proxies) each request to whichever member is available.

Core reverse proxy / load-balancing configuration

```
upstream backend_pool {  
    server 127.0.0.1:3001;  
    server 127.0.0.1:3002;  
}  
  
server {  
    listen 80;  
    server_name _;  
  
    location / {  
        proxy_pass http://backend_pool;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
}
```

The upstream backend pool groups our two backends under one name; NGINX's default balancing method (round robin) alternates between them for each new request. Proxy pass hands the request to that pool. The two proxy_set_header lines forward the original Host and the real client IP address to the backend, so the backend sees who actually made the request rather than only seeing NGINX.

Validating and applying the configuration

```
sudo nginx -t  
sudo systemctl reload nginx
```

```

i@hirwe@ihrwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ cat /etc/nginx/sites-available/the_rooters_web_summative
upstream backend_pool {
    server 127.0.0.1:3001;
    server 127.0.0.1:3002;
}

server {
    listen 80;
    server_name _;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name _;

    location / {
        proxy_pass http://backend_pool;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
i@hirwe@ihrwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$

```

Figure 4: Our NGINX upstream and proxy configuration file (sites-available).

```

i@hirwe@ihrwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-09-12 11:34:02 +04; 10h ago
     Docs: man:nginx(8)
   Process: 10583 ExecReload=/usr/sbin/nginx -g daemon on; master_process on; -s reload (code=exited, status=0/SUCCESS)
  Main PID: 5527 (nginx)
    Tasks: 5 (limit: 18758)
   Memory: 5.3M (peak: 8.5M)
      CPU: 406ms
   CGroup: /system.slice/nginx.service
           └─ 5527 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
              └─ 10585 "nginx: worker process"
                 └─ 10586 "nginx: worker process"
                    └─ 10587 "nginx: worker process"
                       └─ 10588 "nginx: worker process"

Sep 12 11:34:02 i@hirwe-HP-EliteBook-840-G5 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Sep 12 11:34:02 i@hirwe-HP-EliteBook-840-G5 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
Sep 12 13:03:44 i@hirwe-HP-EliteBook-840-G5 systemd[1]: Reloading nginx.service - A high performance web server and a reverse proxy server.
Sep 12 13:03:44 i@hirwe-HP-EliteBook-840-G5 nginx[8922]: 2026/09/12 13:03:44 [notice] 8922#8922: signal process started
Sep 12 13:03:44 i@hirwe-HP-EliteBook-840-G5 systemd[1]: Reloaded nginx.service - A high performance web server and a reverse proxy server.
Sep 12 20:55:37 i@hirwe-HP-EliteBook-840-G5 systemd[1]: Reloading nginx.service - A high performance web server and a reverse proxy server.
Sep 12 20:55:37 i@hirwe-HP-EliteBook-840-G5 nginx[10583]: 2026/09/12 20:55:37 [notice] 10583#10583: signal process started
Sep 12 20:55:37 i@hirwe-HP-EliteBook-840-G5 systemd[1]: Reloaded nginx.service - A high performance web server and a reverse proxy server.
i@hirwe@ihrwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

```

Figure 5: `sudo nginx -t` reporting the configuration syntax as OK before reloading.

Proving load balancing works

We sent repeated requests through NGINX and recorded which backend answered each time:

```
Curl -k https://localhost; echo
```

```

ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to the server 2, of our group the_rooters
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to the server 2, of our group the_rooters
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to server 1 of our group the_rooters.

```

Figure 6: Repeated requests through NGINX, alternating between the Server 1 and Server 2 responses.

Reverse proxy vs. contacting a backend directly

Contacting a backend directly (e.g. `curl http://127.0.0.1:3001`) bypasses NGINX entirely; no load balancing, no HTTPS, no firewall-enforced entry point. Going through the reverse proxy (`curl http://localhost`, and later `https://localhost`) means every request is subject to the same protections and distribution logic, regardless of which backend eventually serves it.

One-backend availability test

With Backend 2 stopped, we repeated the same test and confirmed the service kept responding using Backend 1 alone, then restarted Backend 2 and confirmed both were serving again.

```

ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ hostname -I
172.17.0.154
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ for i in {1..6}; do curl -sk https://localhost; echo; done
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ for i in {1..6}; do curl -sk https://localhost; echo; done
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ ss -ltnp | grep -E '3001|3002'
LISTEN 0      5            127.0.0.1:3001      0.0.0.0:*        users:(("python3",pid=7927,fd=3))

```

Figure 7a: One-backend test: Backend 2 stopped, NGINX still serving via Backend 1 alone.

```

Memory: 5.3M (peak: 8.5M)
CPU: 406ms
CGroup: /system.slice/nginx.service
├─5527 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
├─10585 "nginx: worker process"
├─10586 "nginx: worker process"
├─10587 "nginx: worker process"
└─10588 "nginx: worker process"

Sep 12 11:34:02 ihirwe-HP-EliteBook-840-G5 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server..
Sep 12 11:34:02 ihirwe-HP-EliteBook-840-G5 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
Sep 12 13:03:44 ihirwe-HP-EliteBook-840-G5 systemd[1]: Reloading nginx.service - A high performance web server and a reverse proxy server.
Sep 12 13:03:44 ihirwe-HP-EliteBook-840-G5 nginx[8922]: 2026/09/12 13:03:44 [notice] 8922#8922: signal process started
Sep 12 13:03:44 ihirwe-HP-EliteBook-840-G5 systemd[1]: Reloaded nginx.service - A high performance web server and a reverse proxy server.
Sep 12 20:55:37 ihirwe-HP-EliteBook-840-G5 systemd[1]: Reloading nginx.service - A high performance web server and a reverse proxy server.
Sep 12 20:55:37 ihirwe-HP-EliteBook-840-G5 nginx[10583]: 2026/09/12 20:55:37 [notice] 10583#10583: signal process started
Sep 12 20:55:37 ihirwe-HP-EliteBook-840-G5 systemd[1]: Reloaded nginx.service - A high performance web server and a reverse proxy server.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ ss -ltnp | grep -E '3001|3002'
LISTEN 0      5          127.0.0.1:3001      0.0.0.0:*      users:((("python3",pid=7927,fd=3))
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ ss -ltnp | grep -E '3001|3002'
LISTEN 0      5          127.0.0.1:3002      0.0.0.0:*      users:((("python3",pid=11773,fd=3))
LISTEN 0      5          127.0.0.1:3001      0.0.0.0:*      users:((("python3",pid=7927,fd=3))
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl http://127.0.0.1:3002; echo
Welcome to the server 2, of our group the_rooters
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ for i in {1..8}; do curl -sk https://localhost; echo; done
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to the server 2, of our group the_rooters
Welcome to server 1 of our group the_rooters.
Welcome to the server 2, of our group the_rooters
Welcome to server 1 of our group the_rooters.
Welcome to the server 2, of our group the_rooters
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$

```

Figure 7b: Backend 2 restored , both ports listening again and both servers responding.

What this proves: Requests reach both backends through NGINX in normal operation, and the service continues to respond correctly when one backend is temporarily unavailable, this is the availability benefit load balancing provides.

5. SSL/TLS and HTTPS

HTTP sends data in plain, readable text. HTTPS is HTTP carried inside a TLS-encrypted connection, so data between the client and NGINX cannot be read or tampered with in transit. Since a paid, publicly trusted certificate is not required for this assignment, we generated a self-signed certificate and private key locally.

Certificate and key generation

```

sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/the_rooters.key \
  -out /etc/ssl/certs/the_rooters.crt \
  -subj "/CN=localhost" \
  -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"

```

```
sudo chmod 600 /etc/ssl/private/the_rooters.key
```

-x509 creates a self-signed certificate rather than a request to send to an external authority. -newkey rsa:2048 generates a new 2048-bit private key alongside it. -subj "/CN=localhost" sets the certificate's identity (Common Name) directly. The subjectAltName extension lists both localhost and 127.0.0.1 so the certificate matches however we address the server during testing. chmod 600 restricts the private key so only the root user can read it.

```
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/private/the_rooters.key -out /etc/ssl/certs/the_rooters.crt -subj "/CN=localhost" -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
+++++
.....
+++++
-----
```

Figure 8: Certificate and private key generation command and output.

Certificate identity, issuer and validity

```
openssl x509 -in /etc/ssl/certs/the_rooters.crt -noout -subject -issuer -dates
```

```
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ openssl x509 -in /etc/ssl/certs/the_rooters.crt -noout -subject -issuer -dates
subject=CN = localhost
issuer=CN = localhost
notBefore=Sep 12 16:43:49 2026 GMT
notAfter=Sep 12 16:43:49 2027 GMT
```

Figure 9: Certificate Subject, Issuer, and validity dates (notBefore / notAfter).

Subject identifies who the certificate represents (Common Name: localhost). Issuer identifies who vouched for that identity — for a self-signed certificate, issuer and subject are the same, which is exactly why browsers show a trust warning: there is no independent, publicly trusted authority confirming the identity, only the server vouching for itself. notBefore and notAfter mark the 365-day validity window we generated.

Full NGINX HTTPS configuration

```
upstream backend_pool {
    server 127.0.0.1:3001;
    server 127.0.0.1:3002;
}
```

```

server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name _;

    ssl_certificate /etc/ssl/certs/the_rooters.crt;
    ssl_certificate_key /etc/ssl/private/the_rooters.key;

    location / {
        proxy_pass http://backend_pool;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

```

The first server block now only redirects: any request on port 80 receives a 301 ("permanently moved") response pointing at the same host and path on HTTPS. The second server block listens on 443 with ssl enabled, points at our certificate and private key, and proxies to the same backend_pool used in Section 4; so HTTPS traffic still gets the same reverse-proxying and load balancing.

```

sudo nginx -t
sudo systemctl reload nginx

```

```

ihrwe@ihrwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ cat /etc/nginx/sites-available/the_rooters_web_summative
upstream backend_pool {
    server 127.0.0.1:3001;
    server 127.0.0.1:3002;
}

server {
    listen 80;
    server_name _;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name _;

    ssl_certificate /etc/ssl/certs/the_rooters.crt;
    ssl_certificate_key /etc/ssl/private/the_rooters.key;

    location / {
        proxy_pass http://backend_pool;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

ihrwe@ihrwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$

ihrwe@ihrwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ihrwe@ihrwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ sudo systemctl reload nginx

```

Figure 10: NGINX HTTPS server block configuration.

Testing HTTPS and the redirect

```

curl -k https://localhost
curl -I http://localhost
curl -kL http://localhost

```

-k allows curl to proceed despite the self-signed certificate not being independently trusted — the same warning a browser would show. -I shows headers only, confirming the 301 status and a Location header pointing at https://localhost/. -kL follows the redirect automatically and shows the final HTTPS content in one step.

```

ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ 
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to the server 2, of our group the_rooters
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to the server 2, of our group the_rooters
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost; echo
Welcome to server 1 of our group the_rooters.

```

```

ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -kL http://localhost; echo
Welcome to the server 2, of our group the_rooters
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -kL http://localhost; echo
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -kL http://localhost; echo
Welcome to the server 2, of our group the_rooters
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -kL http://localhost; echo
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -kL http://localhost; echo
Welcome to the server 2, of our group the_rooters
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -kL http://localhost; echo
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -kL http://localhost; echo
Welcome to the server 2, of our group the_rooters
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -kL http://localhost; echo
Welcome to server 1 of our group the_rooters.

```

Figure 11: Successful HTTPS response via `curl -k` and `-kL`

```

ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -I http://localhost; echo
HTTP/1.1 301 Moved Permanently
Server: nginx/1.24.0 (Ubuntu)
Date: Sat, 12 Sep 2026 16:58:04 GMT
Content-Type: text/html
Content-Length: 178
Connection: keep-alive
Location: https://localhost/

```

Figure 12: HTTP request returning a 301 redirect with a Location header pointing to HTTPS.

Why this matters

- HTTPS vs HTTP: HTTPS encrypts the request and response bodies; HTTP does not.
- What TLS provides here: confidentiality of data in transit between client and NGINX, plus a certificate-based identity for the server.

- Certificate vs private key: the certificate is the public identity that can be shared; the private key must remain secret, since it is what allows NGINX to prove it holds the identity in the certificate. It is never included in our submission.
- Why port 443: it is the standard port browsers and clients use automatically for HTTPS.
- Why the self-signed warning appears: there is no external, trusted authority vouching for the certificate.
- Why redirect HTTP to HTTPS: so a visitor who types the plain address, or an old bookmark, is still moved onto the encrypted connection automatically rather than left on an unprotected one.

What this proves: *The site is reachable over HTTPS on port 443 using our certificate, HTTP requests on port 80 are redirected to HTTPS, and the certificate's identity, issuer and validity period match what we generated.*

6. Firewall Configuration and Verification

UFW was configured as the host firewall using the principle of least exposure. The goal was to permit only the services required for administration and normal web access while keeping the backend application services behind NGINX. Ports 22, 80 and 443 therefore have explicit entry-point roles, while ports 3001 and 3002 remain local backend targets.

6.1 Firewall State Before Configuration

Before applying the firewall policy, the existing UFW state was recorded to create a clear baseline.

```
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ sudo ufw status verbose
[sudo] password for ihirwe:
Status: inactive
```

```
ihirwe@ihirwe-HP-EliteBook-840-G5:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)
```

What this proves: The command `sudo ufw status verbose` reports `Status: inactive`, establishing that UFW was not enforcing an active host firewall policy before configuration.

6.11 TCP ports are listening and which processes are using

The **ss -ltnp** command was used to inspect the TCP ports currently listening on the machine and identify the processes using them. This verifies that NGINX is listening on the public web ports 80 and 443, while Backend 1 and Backend 2 are listening only on the loopback address **127.0.0.1** on ports 3001 and 3002.

```
thirwe@thirwe-HP-EliteBook-840-G5: $ ss -ltnp
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	4096	127.0.0.0:53	0.0.0.0:*	
LISTEN	0	5	127.0.0.1:3001	0.0.0.0:*	users(("python3",pid=9199,fd=3))
LISTEN	0	5	127.0.0.1:3002	0.0.0.0:*	users(("python3",pid=15024,fd=3))
LISTEN	0	4096	127.0.0.1:631	0.0.0.0:*	
LISTEN	0	4096	127.0.0.54:53	0.0.0.0:*	
LISTEN	0	511	0.0.0.0:443	0.0.0.0:*	
LISTEN	0	511	0.0.0.0:80	0.0.0.0:*	
LISTEN	0	4096	:::1:631	:::1:*	

What this proves: Ports 80 and 443 are listening for web traffic, while ports 3001 and 3002 are bound to **127.0.0.1**. This confirms that the backend services are locally bound and are intended to be accessed through NGINX rather than directly through the machine's external network interface.

6.2 Final Rules and Required-Port Justification

The **ss -ltnp** command was used to verify which TCP ports were listening and the addresses they were bound to, while **hostname -I** displayed the IP address assigned to the machine. The results show that NGINX listens on ports 80 and 443, while Backend 1 and Backend 2 listen on **127.0.0.1:3001** and **127.0.0.1:3002**. This confirms that the backend services are bound to the local loopback interface rather than the machine's external network address and are intended to be accessed through NGINX.

```
thirwe@thirwe-HP-EliteBook-840-G5: $ ss -ltnp | grep -E '3001|3002'
```

LISTEN	0	5	127.0.0.1:3001	0.0.0.0:*	users(("python3",pid=9199,fd=3))
LISTEN	0	5	127.0.0.1:3002	0.0.0.0:*	users(("python3",pid=15024,fd=3))

```
thirwe@thirwe-HP-EliteBook-840-G5: $ hostname -I
```

172.17.0.180

Ports 3001 and 3002 are bound to **127.0.0.1**, while the machine's network IP is **172.17.0.180**. This confirms that the backend services are locally bound and are not listening directly on the machine's external network interface.

The final policy allows SSH on port 22 for administration, HTTP on port 80 so NGINX can redirect insecure requests to HTTPS, and HTTPS on port 443 as the main secure web entry point. Backend ports 3001 and 3002 are not added to the public UFW allow list.

```

firewall is active and enabled on system startup
thru@thruw-HP-EltteBook-840-G5:~/thru_rooters_web_summative$ sudo ufw status numbered
Status: active

To Action From
--
[ 1] 22/tcp ALLOW IN Anywhere
[ 2] 80/tcp ALLOW IN Anywhere
[ 3] 443/tcp ALLOW IN Anywhere
[ 4] 22/tcp (v6) ALLOW IN Anywhere (v6)
[ 5] 80/tcp (v6) ALLOW IN Anywhere (v6)
[ 6] 443/tcp (v6) ALLOW IN Anywhere (v6)

```

What this proves: UFW is active and the allow list contains only the required entry ports 22, 80 and 443 (plus their IPv6 equivalents). There are no allow rules for ports 3001 or 3002.

Port	Purpose	Security reasoning
22/tcp	SSH administration	Required for system administration; allowed before enabling UFW to avoid lockout.
80/tcp	HTTP redirect	Allows NGINX to receive HTTP and redirect the client to HTTPS.
443/tcp	HTTPS	Primary secure client entry point.
3001/3002	Backend services	Not intentionally exposed publicly; NGINX reaches these local services on loopback.

6.3 Allowed Connection Verification

The required web paths were tested after UFW was enabled to prove that the security policy did not block legitimate traffic.

```
[ 4] 443/tcp (v6)          ALLOW IN    Anywhere (v6)

thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost
Welcome to the server 2, of our group the_rooters!thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -k https://localhost ; echo
Welcome to server 1 of our group the_rooters.
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl -I http://localhost; echo
HTTP/1.1 301 Moved Permanently
Server: nginx/1.24.0 (Ubuntu)
Date: Sat, 12 Sep 2026 17:36:14 GMT
Content-Type: text/html
Content-Length: 178
Connection: keep-alive
Location: https://localhost/

thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ ss -ltnp
```

What this proves: HTTPS successfully returns a backend response, while HTTP returns `301 Moved Permanently` with `Location: https://localhost/`. This verifies that the required web entry points remain functional.

6.4 Backend Ports Are Not Public Entry Points

We also checked the firewall policy together with the backend listening addresses.

```
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ sudo ufw status numbered
ss -ltnp | grep -E '3001|3002'
[sudo] password for thirwe:
Status: active

      To      Action      From
      --      -
[ 1] 22/tcp    ALLOW IN    Anywhere
[ 2] 80/tcp    ALLOW IN    Anywhere
[ 3] 443/tcp    ALLOW IN    Anywhere
[ 4] 22/tcp (v6) ALLOW IN    Anywhere (v6)
[ 5] 80/tcp (v6) ALLOW IN    Anywhere (v6)
[ 6] 443/tcp (v6) ALLOW IN    Anywhere (v6)

LISTEN 0      5          127.0.0.1:3002  0.0.0.0:*    users:(((python3",pid=11773,fd=3)))
LISTEN 0      5          127.0.0.1:3001  0.0.0.0:*    users:(((python3",pid=7927,fd=3)))
thirwe@thirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$
```

What this proves: Ports 3001 and 3002 are absent from the UFW allow list, and `ss -ltnp` shows the Python services bound to `127.0.0.1`. This supports the intended design: clients use NGINX, while NGINX communicates locally with the backend pool.

6.5 Firewall Verification Conclusion

The **ss -ltnp** command was used to verify which TCP ports were listening and the addresses they were bound to, while **hostname -I** displayed the IP address assigned to the machine. The results show that NGINX listens on ports 80 and 443, while Backend 1 and Backend 2 listen on **127.0.0.1:3001** and **127.0.0.1:3002**. This confirms that the backend services are bound to the local loopback interface rather than the machine's external network address and are intended to be accessed through NGINX.

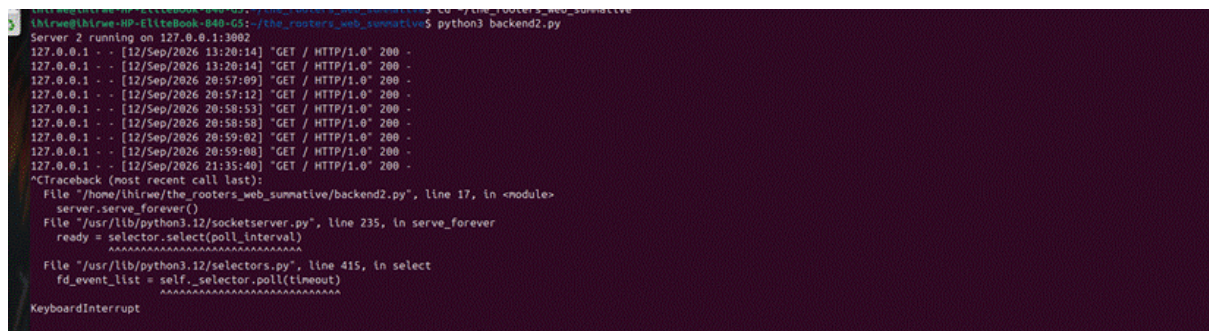
```
thirwe@thirwe-HP-EliteBook-840-G5:~$ ss -ltnp | grep -E '3001|3002'
LISTEN 0      5          127.0.0.1:3001  0.0.0.0:*    users:(("python3",pid=9199,fd=3))
LISTEN 0      5          127.0.0.1:3002  0.0.0.0:*    users:(("python3",pid=15024,fd=3))
thirwe@thirwe-HP-EliteBook-840-G5:~$ hostname -I
172.17.0.180
```

The final firewall configuration follows least exposure: only ports 22, 80 and 443 are permitted for the required administration and web functions. Backend ports 3001 and 3002 are not included in the UFW allow rules and are bound to 127.0.0.1. Successful HTTPS access and HTTP-to-HTTPS redirection confirm that legitimate traffic still works after the firewall is enabled. Together, these tests convincingly prove the intended firewall design.

7. Troubleshooting and Diagnostic Reasoning

The deliberate infrastructure failure was Backend 2 becoming unavailable. Because the infrastructure normally uses two backends behind NGINX, this creates a meaningful availability fault without causing a complete outage. The investigation follows five stages: problem and symptom, evidence, investigation, root cause, and fix with verification.

7.1 Deliberate Fault Introduction



```
lhirwe@lhirwe-HP-EliteBook-840-G5: /the_rooters_web_summative$ cd /the_rooters_web_summative
Server 2 running on 127.0.0.1:3002
127.0.0.1 - - [12/Sep/2026 13:20:14] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 13:20:14] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 20:57:09] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 20:57:12] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 20:58:53] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 20:58:58] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 20:59:02] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 20:59:08] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [12/Sep/2026 21:35:40] "GET / HTTP/1.0" 200 -
^CTraceback (most recent call last):
  File "/home/lhirwe/the_rooters_web_summative/backend2.py", line 17, in <module>
    server.serve_forever()
  File "/usr/lib/python3.12/socketserver.py", line 235, in serve_forever
    ready = selector.select(poll_interval)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/selectors.py", line 415, in select
    fd_event_list = self._selector.poll(timeout)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt
```

What this proves: The terminal shows `backend2.py` running on `127.0.0.1:3002` and then ending with `KeyboardInterrupt` after Ctrl+C. This records the deliberate creation of the fault.

7.2 Stage 1 - Problem and Symptom

After Backend 2 is stopped, repeated HTTPS requests through NGINX continue to work, but only the Server 1 response appears. The service is therefore partially available, but the expected two-backend behaviour has been lost.

```
LISTEN 0 4096 [:::]:631 [::]:*
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ hostname -I
172.17.0.154
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ for i in {1..6}; do curl -sk https://localhost; echo; done
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ for i in {1..6}; do curl -sk https://localhost; echo; done
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ ss -ltnp | grep -E '3001|3002'
LISTEN 0 5 127.0.0.1:3001 0.0.0.0:* users:((("python3",pid=7927,fd=3))
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$
```

What this must prove: Repeated HTTPS requests return only Server 1 and port 3002 is no longer listening. This records the symptom before assuming the cause.

7.3 Stage 2 - Evidence

The backends are then tested directly. Backend 1 should answer on port 3001, while the direct request to port 3002 should fail. This separates the healthy backend from the failing backend.

```
LISTEN 0 5 127.0.0.1:3001 0.0.0.0:* users:((("python3",pid=7927,fd=3))
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl http://127.0.0.1:3001
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl http://127.0.0.1:3001; echo
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ curl http://127.0.0.1:3002; echo
curl: (7) Failed to connect to 127.0.0.1 port 3002 after 0 ms: Couldn't connect to server
```

What this must prove: Backend 1 is reachable, but Backend 2 fails to accept a direct connection. This provides component-level evidence that the failure is specific to Backend 2.

7.4 Stage 3 - Investigation and Narrowing the Cause

The investigation checks NGINX before changing any configuration. ``sudo systemctl status nginx`` verifies the service state, ``sudo nginx -t`` validates the configuration, and ``ss -ltnp | grep -E '3001|3002'`` identifies which backend ports have listening processes.

```
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-09-12 11:34:02 +04; 10h ago
     Docs: man:nginx(8)
  Process: 10583 ExecReload=/usr/sbin/nginx -g daemon on; master_process on; -s reload (code=exited, status=0/SUCCESS)
 Main PID: 5527 (nginx)
    Tasks: 5 (limit: 18758)
  Memory: 5.3M (peak: 8.5M)
     CPU: 406ms
    CGroup: /system.slice/nginx.service
            └─ 5527 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
               └─ 10585 "nginx: worker process"
                  └─ 10586 "nginx: worker process"
                     └─ 10587 "nginx: worker process"
                        └─ 10588 "nginx: worker process"

Sep 12 11:34:02 ihirwe-HP-EliteBook-840-G5 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Sep 12 11:34:02 ihirwe-HP-EliteBook-840-G5 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
Sep 12 13:03:44 ihirwe-HP-EliteBook-840-G5 systemd[1]: Reloading nginx.service - A high performance web server and a reverse proxy server...
Sep 12 13:03:44 ihirwe-HP-EliteBook-840-G5 nginx[8922]: 2026/09/12 13:03:44 [notice] 8922#8922: signal process started
Sep 12 13:03:44 ihirwe-HP-EliteBook-840-G5 systemd[1]: Reloaded nginx.service - A high performance web server and a reverse proxy server.
Sep 12 20:55:37 ihirwe-HP-EliteBook-840-G5 systemd[1]: Reloading nginx.service - A high performance web server and a reverse proxy server...
Sep 12 20:55:37 ihirwe-HP-EliteBook-840-G5 nginx[10583]: 2026/09/12 20:55:37 [notice] 10583#10583: signal process started
Sep 12 20:55:37 ihirwe-HP-EliteBook-840-G5 systemd[1]: Reloaded nginx.service - A high performance web server and a reverse proxy server.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

What this must prove: NGINX remains active and ``nginx -t`` succeeds, while port 3002 has no listener. This rules out an NGINX service/configuration failure and convincingly narrows the fault to Backend 2.

NGINX is running and its configuration is valid, while Backend 1 remains reachable. Backend 2 fails its direct connection test and port 3002 has no listening process. The evidence therefore supports investigating the Backend 2 process rather than changing NGINX or the firewall.

7.5 Stage 4 - Root Cause

The root cause was the stopped Backend 2 Python process. Once that process ended, nothing was listening on `127.0.0.1:3002`. NGINX therefore had only Backend 1 available. This conclusion is supported by the failed direct request to port 3002, the missing listener, and the successful NGINX checks.

```
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ ss -ltnp | grep -E '3001|3002'
LISTEN 0      5            127.0.0.1:3001      0.0.0.0:*        users:((("python3",pid=7927,fd=3))
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ ss -ltnp | grep -E '3001|3002'
LISTEN 0      5            127.0.0.1:3002      0.0.0.0:*        users:((("python3",pid=11773,fd=3))
LISTEN 0      5            127.0.0.1:3001      0.0.0.0:*        users:((("python3",pid=7927,fd=3))
```

7.6 Stage 5 - Fix and Verification

Backend 2 was restarted using its normal Python3 backend2.py start command. Recovery must be verified both at the backend level and through the complete HTTPS/NGINX path.

```
KeyboardInterrupt
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ python3 backend2.py
Server 2 running on 127.0.0.1:3002
```

```
ihirwe@ihirwe-HP-EliteBook-840-G5:~$ curl http://127.0.0.1:3001
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~$ curl http://127.0.0.1:3002
Welcome to the server 2, of our group the_rooters
ihirwe@ihirwe-HP-EliteBook-840-G5:~$ ss -ltnp | grep -E '3001|3002'
LISTEN 0      5            127.0.0.1:3002      0.0.0.0:*        users:(("python3",pid=7933,fd=3))
LISTEN 0      5            127.0.0.1:3001      0.0.0.0:*        users:(("python3",pid=7927,fd=3))
ihirwe@ihirwe-HP-EliteBook-840-G5:~$
```

What this proves: Direct requests to ports 3001 and 3002 return their distinguishable responses, and `ss -ltnp` shows both loopback listeners. This confirms that the Backend 2 process has been restored.

```
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ ss -ltnp | grep -E '3001|3002'
LISTEN 0      5            127.0.0.1:3002      0.0.0.0:*        users:(("python3",pid=15092,fd=3))
LISTEN 0      5            127.0.0.1:3001      0.0.0.0:*        users:(("python3",pid=7927,fd=3))
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$ for i in {1..8}; do curl -sk https://localhost; echo; done
Welcome to the server 2, of our group the_rooters
Welcome to server 1 of our group the_rooters.
Welcome to the server 2, of our group the_rooters
Welcome to server 1 of our group the_rooters.
Welcome to the server 2, of our group the_rooters
Welcome to server 1 of our group the_rooters.
Welcome to the server 2, of our group the_rooters
Welcome to server 1 of our group the_rooters.
ihirwe@ihirwe-HP-EliteBook-840-G5:~/the_rooters_web_summative$
```

What this must prove: `ss -ltnp | grep -E '3001|3002'` and `for i in {1..8}; do curl -sk https://localhost; echo; done` show both ports listening and repeated HTTPS responses from both Server 1 and Server 2. As strongest proof that the complete load-balanced service is done.

7.7 Troubleshooting Outcome

The troubleshooting process did not jump from symptom to fix. The group reproduced the problem, collected direct evidence, checked and ruled out NGINX, narrowed the fault to the missing Backend 2 listener, identified the stopped process as the root cause, restarted it, and verified recovery. The continued availability of Server 1 while Backend 2 was down also demonstrates the resilience benefit of multiple backends.

9. Sources, Credits and AI Transparency

The group used Anthropic's Claude (an AI assistant) as a supplementary learning aid during this assignment: to explain NGINX, TLS/certificate and UFW directives before we used them. All commands, configuration files, screenshots and test results presented in this report were produced by running our own commands in our own Linux environment; each group member is able to explain the configuration and reasoning behind the sections of this report.

Anthropic. (2026). Claude (Sonnet) [Large language model]. <https://claude.ai>